

LAPORAN PRATIKUM
STRUKTUR DATA
“LinkedList”



Dosen Pengampu:
DR. Wahyudi. .S.T.M.T

Asisten Lab:
Jovantri Immanuel Gulo

Disusun Oleh:
Marcello Bayu Denar Widjaksono
2511532011

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “LinkedList”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan program komputer, pemahaman mendalam tentang struktur data (data structure) menjadi fondasi utama bagi para programmer. Salah satu bentuk struktur data yang paling sering digunakan adalah Abstract Data Type (ADT), yang berfungsi mengorganisir data agar mudah diakses, dimanipulasi, dan dikelola untuk meningkatkan kinerja aplikasi. Pada bahasa pemrograman Java, implementasi ADT List berbasis tautan dapat dilakukan melalui kelas `LinkedList` yang tersedia dalam paket `java.util`. `LinkedList` merupakan struktur data linear yang terdiri dari rangkaian node (simpul) yang saling terhubung melalui referensi, di mana setiap node menyimpan nilai data serta alamat ke node berikutnya (dan sebelumnya pada implementasi `DoublyLinkedList`). Perbedaan mendasar antara `LinkedList` dengan struktur data linear berbasis indeks seperti `Array` terletak pada alokasi memori dan mekanisme akses elemen: `LinkedList` tidak memerlukan blok memori yang bersebelahan dan ukurannya bersifat dinamis, sehingga operasi penyisipan serta penghapusan elemen menjadi sangat efisien tanpa perlu menggeser data lain, meskipun akses acak memerlukan penelusuran sekuensial dari node awal. Karakteristik ini membuat `LinkedList` sangat fleksibel dan banyak diterapkan dalam berbagai skenario nyata, seperti implementasi riwayat navigasi (browser history), fitur undo/redo pada aplikasi, manajemen playlist, representasi graf melalui adjacency list, serta pembuatan cache dengan algoritma LRU (Least Recently Used).

1.2 Tujuan Pratikum

1. Menjelaskan konsep dan karakteristik `LinkedList` sebagai implementasi struktur data ADT berbasis node yang terhubung secara linear dalam Java.
2. Mengidentifikasi metode-metode utama `LinkedList` (seperti `add`, `remove`, `get`, `set`, `peek`, dan `size`) beserta mekanisme kerjanya.

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman konsep struktur data dinamis dan non-kontigu melalui penggunaan LinkedList dalam bahasa pemrograman Java.
2. Mampu menggunakan metode-metode utama LinkedList untuk menyelesaikan permasalahan komputasi yang relevan..

BAB II PEMBAHASAN

2.1 Pratikum Striktur Data

```
package pekan5_2511532011;

public class NodeSLL_2511532011 {

    // node bagian data
    int data_2011;
    // pointer ke node berikutnya
    NodeSLL_2511532011 next_2011;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532011(int data_2011) {
        this.data_2011 = data_2011;
        this.next_2011 = null;
    }
}
```

Pembahasan Class NodeSLL:

1. Atribut Data (data_2011)

int data_2011;; Deklarasi variabel instance bertipe integer untuk menyimpan nilai/data yang dibawa oleh node ini.

Dalam konteks SLL, atribut ini adalah payload atau muatan informasi yang ingin disimpan dalam struktur data.

Akses modifier default (package-private) memungkinkan kelas lain dalam paket yang sama untuk mengaksesnya secara langsung.

2. Atribut Pointer (next_2011)

NodeSLL_2011 next_2011;; Deklarasi variabel instance bertipe referensi ke kelas NodeSLL_2511532011 itu sendiri.

Atribut ini berfungsi sebagai pointer atau penunjuk yang menghubungkan node saat ini ke node berikutnya dalam rangkaian linked list.

Konsep ini disebut self-referential class, dimana sebuah kelas memiliki atribut yang bertipe sama dengan kelas itu sendiri, yang menjadi dasar pembentukan struktur berantai.

3. Konstruktor Node

`public NodeSLL_2511532011(int data_2011):` Mendefinisikan konstruktor publik yang menerima satu parameter integer untuk inisialisasi node. Konstruktor ini dipanggil setiap kali ingin membuat node baru dalam linked list.

4. Inisialisasi Data dalam Konstruktor

`this.data_2011 = data_2011;` Mengassign nilai parameter `data_2011` ke variabel instance `this.data_2011`.

Keyword `this` digunakan untuk membedakan antara atribut kelas dengan parameter metode yang memiliki nama sama.

5. Inisialisasi Pointer dalam Konstruktor

`this.next_2011 = null;` Mengassign nilai `null` ke atribut pointer `next_2011`.

Penting: Inisialisasi `null` menandakan bahwa node baru ini belum terhubung ke node lain.

Dalam SLL, node dengan `next = null` biasanya merepresentasikan node terakhir (tail) atau node yang berdiri sendiri (single element).

```

package pekan5_2511532011;

public class TambahSLL_2511532011 {

    public static NodeSLL_2511532011 insertAtFront_2011(NodeSLL_2511532011 head_2011, int value_2011) {
        NodeSLL_2511532011 new_node_2011 = new NodeSLL_2511532011(value_2011);
        new_node_2011.next_2011 = head_2011;
        return new_node_2011;
    }

    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511532011 insertAtEnd_2011(NodeSLL_2511532011 head_2011, int value_2011) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511532011 newNode_2011 = new NodeSLL_2511532011(value_2011);
        // jika list kosong maka node jadi head
        if (head_2011 == null) {
            return newNode_2011;
        }

        // simpan head ke variabel sementara
        NodeSLL_2511532011 last_2011 = head_2011;
        // telusuri ke node akhir
        while (last_2011.next_2011 != null) {
            last_2011 = last_2011.next_2011;
        }
        // ubah pointer
        last_2011.next_2011 = newNode_2011;
        return head_2011;
    }

    static NodeSLL_2511532011 GetNode_2011(int data_2011) {
        return new NodeSLL_2511532011(data_2011);
    }

    static NodeSLL_2511532011 insertPos_2011(NodeSLL_2511532011 headNode_2011, int position_2011, int value_2011) {
        NodeSLL_2511532011 head_2011 = headNode_2011;
        if (position_2011 < 1) {
            System.out.print("Invalid position");
        }
        if (position_2011 == 1) {
            NodeSLL_2511532011 new_node_2011 = new NodeSLL_2511532011(value_2011);
            new_node_2011.next_2011 = head_2011;
            return new_node_2011;
        } else {
            while (position_2011-- != 0) {
                if (position_2011 == 1) {
                    NodeSLL_2511532011 newNode_2011 = GetNode_2011(value_2011);
                    newNode_2011.next_2011 = headNode_2011.next_2011;
                    headNode_2011.next_2011 = newNode_2011;
                    break;
                }
                headNode_2011 = headNode_2011.next_2011;
            }
            if (position_2011 != 1)
                System.out.print("Posisi di luar jangkauan");
        }
        return head_2011;
    }

    public static void printList_2011(NodeSLL_2511532011 head_2011) {
        NodeSLL_2511532011 curr_2011 = head_2011;
        while (curr_2011.next_2011 != null) {
            System.out.print(curr_2011.data_2011 + "-->");
            curr_2011 = curr_2011.next_2011;
        }
        if (curr_2011.next_2011 == null) {
            System.out.print(curr_2011.data_2011);
        }
        System.out.println();
    }

    public static void main (String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(2);
        head_2011.next_2011 = new NodeSLL_2511532011(3);
        head_2011.next_2011.next_2011 = new NodeSLL_2511532011(5);
        head_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(6);

        // cetak list asli
        System.out.print("Senarai berantai awal:");
        printList_2011(head_2011);

        // tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data_2011 = 1;
        head_2011 = insertAtFront_2011(head_2011, data_2011);

        // cetak update list
        printList_2011(head_2011);

        // tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_2011 = 7;
        head_2011 = insertAtEnd_2011(head_2011, data2_2011);

        // cetak update list
        printList_2011(head_2011);
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_2011 = 4;
        int pos_2011 = 4;
        head_2011 = insertPos_2011(head_2011, pos_2011, data3_2011);

        // cetak update list
        printList_2011(head_2011);
    }
}

```


Pembahasan Class TambahSLL:

1. Metode insertAtFront_2011 (Sisip Depan)

- `Public static NodeSLL_2511532011 insertAtFront_2011(NodeSLL_2511532011 head_2011, int value_2011);` Metode statis untuk menambahkan node baru di posisi paling depan (head) list.
- `NodeSLL_2511532011 new_node_2011 = new NodeSLL_2511532011(value_2011);` Membuat instance node baru dengan nilai yang diberikan.
- `new_node_2011.next_2011 = head_2011;` Logika Kunci. Pointer next dari node baru diarahkan ke head lama, sehingga node baru "menunjuk" ke seluruh rangkaian list yang ada.
- `return new_node_2011;` Mengembalikan referensi node baru sebagai head yang updated, karena posisi head telah berubah.

2. Metode insertAtEnd_2011 (Sisip Belakang)

- `public static NodeSLL_2511532011 insertAtEnd_2011(...);` Metode statis untuk menambahkan node baru di posisi paling akhir (tail) list.
- `NodeSLL_2511532011 newNode_2011 = new NodeSLL_2511532011(value_2011);` Membuat node baru dengan nilai yang diberikan.
- `if (head_2011 == null) { return newNode_2011; }` Handle kasus khusus jika list masih kosong, maka node baru langsung menjadi head.
- `NodeSLL_2511532011 last_2011 = head_2011;` Inisialisasi pointer sementara last_2011 yang dimulai dari head untuk traversing.
- `while (last_2011.next_2011 != null) { last_2011 = last_2011.next_2011; }` Loop traversing hingga menemukan node terakhir (node yang next-nya bernilai null).
- `last_2011.next_2011 = newNode_2011;` Menghubungkan node terakhir saat ini ke node baru, sehingga node baru menjadi tail yang baru.
- `return head_2011;` Mengembalikan head asli karena posisi head tidak berubah.

3. Metode GetNode_2011 (Helper)

- `static NodeSLL_2511532011 GetNode_2011(int data_2011):` Metode helper statis untuk membuat dan mengembalikan node baru.
- `return new NodeSLL_2511532011(data_2011);` Instansiasi singkat yang memudahkan pembuatan node di metode lain tanpa menulis kode `new Node...` berulang kali.

4. Metode insertPos_2011 (Sisip Posisi Tertentu)

- `static NodeSLL_2511532011 insertPos_2011(...):` Metode untuk menyisipkan node pada posisi index tertentu (1-based index).
- `NodeSLL_2511532011 head_2011 = headNode_2011;` Menyimpan referensi head awal untuk dikembalikan nanti.
- `if (position_2011 < 1) { System.out.print("Invalid position"); }` Validasi posisi tidak boleh kurang dari 1.
- `if (position_2011 == 1) { ... }` Kasus khusus jika posisi = 1 (sama dengan insert di depan), buat node baru dan arahkan next-nya ke head lama.
- `while (position_2011-- != 0) { ... }` Loop untuk bergerak menuju posisi yang diinginkan dengan decrement counter.
- `if (position_2011 == 1) { ... }` Ketika sudah berada tepat sebelum posisi target:
- `newNode_2011.next_2011 = headNode_2011.next_2011;` Node baru menunjuk ke node yang sebelumnya ada di posisi target.
- `headNode_2011.next_2011 = newNode_2011;` Node saat ini (sebelum target) sekarang menunjuk ke node baru.
- `break;` Keluar dari loop karena penyisipan selesai.
- `headNode_2011 = headNode_2011.next_2011;` Geser pointer traversal ke node berikutnya.
- `if (position_2011 != 1) System.out.print("Posisi di luar jangkauan");` Validasi jika posisi melebihi panjang list.
- `return head_2011;` Kembalikan referensi head.

5. Metode printList_2011 (Cetak List)

- `public static void printList_2011(NodeSLL_2511532011 head_2011):`
Metode untuk menampilkan isi linked list ke konsol.
- `NodeSLL_2511532011 curr_2011 = head_2011;` Inisialisasi pointer `curr` untuk traversing mulai dari `head`.
- `while (curr_2011.next_2011 != null) { ... };` Loop untuk mencetak semua node kecuali node terakhir.
- `System.out.print(curr_2011.data_2011 + "-->");` Cetak nilai data diikuti panah `-->` untuk visualisasi alur list.
- `curr_2011 = curr_2011.next_2011;` Geser pointer ke node berikutnya.
- `if (curr_2011.next_2011 == null)`
`{ System.out.print(curr_2011.data_2011); }` Cetak node terakhir tanpa panah karena merupakan ujung list.
- `System.out.println();` Pindah baris setelah selesai mencetak seluruh list.

6. Metode main - Inisialisasi List Awal

- `NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(2);`
Membuat node pertama dengan nilai 2 sebagai `head`.
- `head_2011.next_2011 = new NodeSLL_2511532011(3);`
Menambahkan node kedua (nilai 3) dengan menghubungkan langsung via pointer.
- `head_2011.next_2011.next_2011 = new NodeSLL_2511532011(5);`
Menambahkan node ketiga (nilai 5).
- `head_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(6);`
Menambahkan node keempat (nilai 6).
- State Awal: List terbentuk: 2 --> 3 --> 5 --> 6 --> null.

7. Metode main - Cetak List Awal

- `System.out.print("Senarai berantai awal:");` Label output.
- `printList_2011(head_2011);` Memanggil metode `print` untuk menampilkan: 2-->3-->5-->6.

8. Metode main - Insert Depan (Nilai 1)

- `int data_2011 = 1;` Siapkan nilai yang akan disisipkan.

- `head_2011 = insertAtFront_2011(head_2011, data_2011);` Panggil metode insert depan. Penting: Hasil return harus di-assign kembali ke `head_2011` karena referensi head berubah.
- `printList_2011(head_2011);` Cetak hasil: 1-->2-->3-->5-->6.

9. Metode main - Insert Belakang (Nilai 7)

- `int data2_2011 = 7;` Siapkan nilai untuk tail.
- `head_2011 = insertAtEnd_2011(head_2011, data2_2011);` Panggil metode insert belakang. Head tetap sama, tapi tail sekarang bernilai 7.
- `printList_2011(head_2011);` Cetak hasil: 1-->2-->3-->5-->6-->7.

10. Metode main - Insert Posisi (Nilai 4 di Posisi 4)

- `int data3_2011 = 4; int pos_2011 = 4;` Siapkan nilai 4 untuk disisipkan di posisi ke-4 (1-based).
- `head_2011 = insertPos_2011(head_2011, pos_2011, data3_2011);` Sisipkan node. Posisi ke-4 sebelumnya adalah nilai 5, jadi 4 akan masuk sebelum 5.
- `printList_2011(head_2011);` Cetak hasil akhir: 1-->2-->3-->4-->5-->6-->7.

```

package pekan5_2511532011;

public class HapusSLL_2511532011 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511532011 deleteHead_2011(NodeSLL_2511532011 head_2011) {
        // jika SLL kosong
        if (head_2011 == null)
            return null;
        // pindahkan head ke node berikutnya
        head_2011 = head_2011.next_2011;
        // return head baru
        return head_2011;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532011 removeLastNode_2011(NodeSLL_2511532011 head_2011) {
        // jika list kosong, return null
        if (head_2011 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head_2011.next_2011 == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511532011 secondLast_2011 = head_2011;
        while (secondLast_2011.next_2011.next_2011 != null) {
            secondLast_2011 = secondLast_2011.next_2011;
        }
        // hapus node terakhir
        secondLast_2011.next_2011 = null;
        return head_2011;
    }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511532011 deleteNode_2011(NodeSLL_2511532011 head_2011, int position_2011) {
        NodeSLL_2511532011 temp_2011 = head_2011;
        NodeSLL_2511532011 prev_2011 = null;
        // jika linked list null
        if (temp_2011 == null)
            return head_2011;
        // kasus 1: head dihapus
        if (position_2011 == 1) {
            head_2011 = temp_2011.next_2011;
            return head_2011;
        }
        // kasus 2: menghapus node di tengah
        // telusuri ke node yang di hapus
        for (int i_2011 = 1; temp_2011 != null && i_2011 < position_2011; i_2011++) {
            prev_2011 = temp_2011;
            temp_2011 = temp_2011.next_2011;
        }
        // jika ditemukan, hapus node
        if (temp_2011 != null) {
            prev_2011.next_2011 = temp_2011.next_2011;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_2011;
    }

    // fungsi mencetak SLL
    public static void printList_2011(NodeSLL_2511532011 head_2011) {
        NodeSLL_2511532011 curr_2011 = head_2011;
        while (curr_2011.next_2011 != null) {
            System.out.print(curr_2011.data_2011+"->");
            curr_2011 = curr_2011.next_2011;
        }
        if (curr_2011.next_2011 == null) {
            System.out.print(curr_2011.data_2011);
        }
        System.out.println();
    }

    // kelas main
    public static void main(String[] args) {
        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(1);
        head_2011.next_2011 = new NodeSLL_2511532011(2);
        head_2011.next_2011.next_2011 = new NodeSLL_2511532011(3);
        head_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(4);
        head_2011.next_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(5);
        head_2011.next_2011.next_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(6);
        // cetak list awal
        System.out.println("List awal: ");
        printList_2011(head_2011);

        //hapus head
        head_2011 = deleteHead_2011(head_2011);
        System.out.println("List setelah head dihapus: ");
        printList_2011(head_2011);

        //hapus node terakhir
        head_2011 = removeLastNode_2011(head_2011);
        System.out.println("List setelah simpul terakhir di hapus: ");
        printList_2011(head_2011);

        // deleting node at position 2
        int position_2011 = 2;
        head_2011 = deleteNode_2011(head_2011, position_2011);

        // print list after deletion
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_2011(head_2011);
    }
}

```

Pembahasan Class HapusSLL:

1. Metode deleteHead_2011 (Hapus Depan)

- `public static NodeSLL_2511532011 deleteHead_2011(NodeSLL_2511532011 head_2011):` Metode statis untuk menghapus node paling depan (head) dari list.
- `if (head_2011 == null) return null;`: Guard clause. Jika list kosong, langsung kembalikan null karena tidak ada yang bisa dihapus.
- `head_2011 = head_2011.next_2011;`: Logika Kunci. Pindahkan referensi head ke node berikutnya. Node lama akan otomatis ter-deallocate oleh garbage collector Java karena tidak ada lagi referensi yang menunjuk ke sana.
- `return head_2011;`: Kembalikan referensi head yang baru karena posisi head telah berubah.

2. Metode removeLastNode_2011 (Hapus Belakang)

- `public static NodeSLL_2511532011 removeLastNode_2011(...):` Metode statis untuk menghapus node paling akhir (tail) dari list.
- `if (head_2011 == null) { return null; }`: Handle kasus list kosong.
- `if (head_2011.next_2011 == null) { return null; }`: Handle kasus list hanya memiliki satu node. Setelah dihapus, list menjadi kosong (null).
- `NodeSLL_2511532011 secondLast_2011 = head_2011;` Inisialisasi pointer untuk menemukan node kedua terakhir.
- `while (secondLast_2011.next_2011.next_2011 != null) { ... }`: Loop traversing hingga menemukan node yang next-nya menunjuk ke tail (node terakhir).
- `secondLast_2011.next_2011 = null;`: Logika Penghapusan. Putuskan hubungan pointer dari node kedua terakhir ke tail. Node tail terisolasi dan akan dihapus oleh garbage collector.
- `return head_2011;`: Kembalikan head asli karena posisi head tidak berubah.

3. Metode deleteNode_2011 (Hapus Posisi Tertentu)

- `public static NodeSLL_2511532011 deleteNode_2011(...)`: Metode untuk menghapus node pada posisi index tertentu (1-based index).
- `NodeSLL_2511532011 temp_2011 = head_2011;`: Pointer temp untuk traversing menuju node target yang akan dihapus.
- `NodeSLL_2511532011 prev_2011 = null;`: Pointer prev untuk menyimpan referensi node sebelum temp, diperlukan untuk menyambung ulang list.
- `if (temp_2011 == null) return head_2011;`: Handle kasus list kosong.
- `if (position_2011 == 1) { head_2011 = temp_2011.next_2011; return head_2011; }`: Kasus khusus jika menghapus head. Langsung pindahkan head ke node berikutnya.
- `for (int i_2011 = 1; temp_2011 != null && i_2011 < position_2011; i_2011++) { ... }`: Loop untuk bergerak menuju posisi target. Setiap iterasi menggeser prev dan temp satu langkah ke depan.
- `prev_2011 = temp_2011; temp_2011 = temp_2011.next_2011;`: Update pointer traversing.
- `if (temp_2011 != null) { prev_2011.next_2011 = temp_2011.next_2011; }`: Logika Penghapusan. Hubungkan node sebelumnya (prev) langsung ke node setelah target (temp.next), sehingga node target terlewat (terhapus dari rangkaian).
- `else { System.out.println("Data tidak ada"); }`: Handle kasus jika posisi melebihi panjang list.
- `return head_2011;`: Kembalikan referensi head.

4. Metode printList_2011 (Cetak List)

- `public static void printList_2011(NodeSLL_2511532011 head_2011)`: Metode untuk menampilkan isi linked list ke konsol.
- `NodeSLL_2511532011 curr_2011 = head_2011;`: Inisialisasi pointer curr untuk traversing mulai dari head.
- `while (curr_2011.next_2011 != null)`
`{ System.out.print(curr_2011.data_2011+"-->"); ... }`: Loop mencetak semua node kecuali terakhir dengan format panah "-->".

- if (curr_2011.next_2011 == null) { System.out.print(curr_2011.data_2011); } : Cetak node terakhir tanpa panah karena merupakan ujung list.
 - System.out.println();: Pindah baris setelah selesai mencetak.
5. Metode main - Inisialisasi List Awal
- NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(1);: Membuat node pertama dengan nilai 1 sebagai head.
 - Rangkaian head_2011.next_2011 = new NodeSLL_2511532011(2); dst.: Membangun list secara manual dengan menghubungkan pointer next antar node.
 - State Awal: List terbentuk: 1 --> 2 --> 3 --> 4 --> 5 --> 6 --> null.
6. Metode main - Cetak List Awal
- System.out.println("list awal: ");: Label output.
 - printList_2011(head_2011);: Memanggil metode print untuk menampilkan: 1-->2-->3-->4-->5-->6.
 - Metode main - Delete Head
 - head_2011 = deleteHead_2011(head_2011);: Panggil metode hapus head. Node dengan nilai 1 dihapus.
 - Penting: Hasil return harus di-assign kembali ke head_2011 karena referensi head berubah ke node bernilai 2.
 - printList_2011(head_2011);: Cetak hasil: 2-->3-->4-->5-->6.
7. Metode main - Delete Last Node
- head_2011 = removeLastNode_2011(head_2011);: Panggil metode hapus tail. Node dengan nilai 6 dihapus.
 - Pointer dari node 5 diubah menjadi null, memutus referensi ke node 6.
 - printList_2011(head_2011);: Cetak hasil: 2-->3-->4-->5.
8. Metode main - Delete Node at Position 2
- int position_2011 = 2;: Tentukan posisi yang akan dihapus (1-based index).
 - head_2011 = deleteNode_2011(head_2011, position_2011);: Hapus node di posisi ke-2. Saat ini list adalah [2,3,4,5], posisi 2 adalah nilai 3.

- Logika: Node 2 (prev) akan langsung menunjuk ke node 4 (temp.next), melewati node 3.
- `printList_2011(head_2011);`: Cetak hasil akhir: 2-->4-->5.

```

package pekan5_2511532011;

public class PencarianSLL_2511532011 {
    static boolean searchKey_2011(NodeSLL_2511532011 head_2011, int key_2011) {
        NodeSLL_2511532011 curr_2011 = head_2011;
        while (curr_2011 != null) {
            if (curr_2011.data_2011 == key_2011)
                return true;
            curr_2011 = curr_2011.next_2011;
        }
        return false;
    }

    public static void traversal_2011(NodeSLL_2511532011 head_2011) {
        // mulai dari head
        NodeSLL_2511532011 curr_2011 = head_2011;
        while (curr_2011 != null) {
            System.out.print(" " + curr_2011.data_2011);
            curr_2011 = curr_2011.next_2011;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(14);
        head_2011.next_2011 = new NodeSLL_2511532011(21);
        head_2011.next_2011.next_2011 = new NodeSLL_2511532011(13);
        head_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(30);
        head_2011.next_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(10);
        System.out.print("Penelusuran SLL : ");
        traversal_2011(head_2011);
        // data yang akan dicari
        int key_2011 = 30;
        System.out.print("cari data " + key_2011 + " = ");
        if (searchKey_2011(head_2011, key_2011))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}

```

Pembahasan Class PencarianSLL:

1. Metode searchKey_2011 (Pencarian Linear)

- static boolean searchKey_2011(NodeSLL_2511532011 head_2011, int key_2011): Metode statis yang menerima referensi head list dan nilai kunci (key) yang ingin dicari, mengembalikan boolean (true jika ditemukan, false jika tidak).
- NodeSLL_2511532011 curr_2011 = head_2011;: Inisialisasi pointer curr yang dimulai dari head untuk traversing list.
- while (curr_2011 != null): Loop berjalan selama pointer curr belum mencapai akhir list (null).
- if (curr_2011.data_2011 == key_2011) return true;: Logika Pencarian. Membandingkan nilai data pada node saat ini dengan key. Jika cocok, langsung kembalikan true dan hentikan pencarian (early return).

- `curr_2011 = curr_2011.next_2011;` Geser pointer curr ke node berikutnya untuk melanjutkan pencarian.
- `return false;` Jika loop selesai tanpa menemukan kecocokan (semua node sudah dicek), kembalikan false menandakan data tidak ada dalam list.
- Kompleksitas Waktu: $O(n)$ dalam worst case karena mungkin harus mengecek seluruh elemen list. Kompleksitas Ruang: $O(1)$ karena hanya menggunakan satu pointer tambahan.

2. Metode traversal_2011 (Penelusuran/Cetak List)

- `public static void traversal_2011(NodeSLL_2511532011 head_2011):` Metode statis untuk menampilkan seluruh elemen list ke konsol secara berurutan.
- `NodeSLL_2511532011 curr_2011 = head_2011;` Inisialisasi pointer curr mulai dari head untuk traversing.
- `while (curr_2011 != null):` Loop berjalan selama masih ada node yang belum diakses.
- `System.out.print(" " + curr_2011.data_2011);` Cetak nilai data node saat ini dengan spasi sebagai pemisah antar elemen.
- `curr_2011 = curr_2011.next_2011;` Geser pointer ke node berikutnya.
- `System.out.println();` Pindah baris setelah seluruh elemen selesai dicetak untuk kerapian output.

3. Metode main - Inisialisasi List Awal

- `NodeSLL_2511532011 head_2011 = new NodeSLL_2511532011(14);` Membuat node pertama dengan nilai 14 sebagai head dari linked list.
- `head_2011.next_2011 = new NodeSLL_2511532011(21);` Menambahkan node kedua dengan nilai 21 dan menghubungkannya via pointer next.
- `head_2011.next_2011.next_2011 = new NodeSLL_2511532011(13);` Menambahkan node ketiga dengan nilai 13.
- `head_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(30);` Menambahkan node keempat dengan nilai 30.

- `head_2011.next_2011.next_2011.next_2011.next_2011 = new NodeSLL_2511532011(10);` Menambahkan node kelima dengan nilai 10 sebagai tail.
 - State List: Terbentuk rangkaian: 14 --> 21 --> 13 --> 30 --> 10 --> null.
4. Metode main - Traversal Awal
- `System.out.print("Penelusuran SLL : ");` Mencetak label untuk output traversal.
 - `traversal_2011(head_2011);` Memanggil metode traversal untuk menampilkan seluruh isi list.
 - Expected Output: Penelusuran SLL : 14 21 13 30 10.
5. Metode main - Persiapan Pencarian
- `int key_2011 = 30;` Mendefinisikan variabel key dengan nilai 30 yang akan dicari dalam list.
 - `System.out.print("cari data " +key_2011+ " = ");` Mencetak prompt informasi tentang data yang sedang dicari.
6. Metode main - Eksekusi Pencarian
- `if (searchKey_2011(head_2011, key_2011));` Memanggil metode `searchKey` dengan parameter head list dan nilai kunci 30.
- Alur Pencarian:
- Iterasi 1: `curr.data = 14`, tidak cocok dengan 30, lanjut.
- Iterasi 2: `curr.data = 21`, tidak cocok dengan 30, lanjut.
- Iterasi 3: `curr.data = 13`, tidak cocok dengan 30, lanjut.
- Iterasi 4: `curr.data = 30`, cocok! Metode langsung `return true`.
- `System.out.println("ketemu");` Karena metode mengembalikan `true`, blok `if` dieksekusi dan mencetak "ketemu".
 - `else { System.out.println("tidak ada"); }` Blok `else` hanya dieksekusi jika metode mengembalikan `false` (data tidak ditemukan).

2.2 Tugas Struktur Data

```
package pekan5_2511532011;

public class Pasien_2511532011 {
    String nama_2011;
    String keluhan_2011;
    int Antrian_2011;

    Pasien_2511532011 next_2011;

    public Pasien_2511532011(int a, String n, String k) {
        this.Antrian_2011 = a;
        this.nama_2011 = n;
        this.keluhan_2011 = k;
        this.next_2011 = null;
    }

    public void setN_2011( String n){
        this.nama_2011 = n;
    }

    public void setK_2011( String k){
        this.keluhan_2011 = k;
    }

    public void setA_2011( int a){
        this.Antrian_2011 = a;
    }

    public String getN_2011() {
        return nama_2011;
    }

    public String getK_2011() {
        return keluhan_2011;
    }

    public int getA_2011() {
        return Antrian_2011;
    }
}
```

Pembahasan Class Pasien:

1. Deklarasi Kelas

public class Pasien_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai cetak biru (blueprint) untuk objek data pasien.

Kelas ini dirancang sebagai node dalam Singly Linked List, sehingga memiliki atribut data dan pointer ke node berikutnya.

2. Atribut Data Pasien

String nama_2011;: Variabel instance untuk menyimpan nama pasien bertipe String.

String keluhan_2011;; Variabel instance untuk menyimpan deskripsi keluhan atau diagnosa pasien bertipe String.

int Antrian_2011;; Variabel instance bertipe integer untuk menyimpan nomor urut antrian pasien.

3. Atribut Pointer Linked List

Pasien_2511532011 next_2011;; Variabel instance bertipe referensi ke kelas Pasien_2511532011 itu sendiri.

Berfungsi sebagai pointer yang menghubungkan objek pasien saat ini ke objek pasien berikutnya dalam rangkaian antrian linked list.

Konsep self-referential class ini adalah fondasi pembentukan struktur data berantai.

4. Konstruktor Kelas

public Pasien_2511532011(int a, String n, String k): Konstruktor publik yang menerima tiga parameter untuk inisialisasi objek pasien baru.

Parameter a: Nilai integer untuk nomor antrian.

Parameter n: String untuk nama pasien.

Parameter k: String untuk keluhan pasien.

Inisialisasi Atribut dalam Konstruktor

this.Antrian_2011 = a;; Assign nilai parameter a ke variabel instance Antrian_2011.

this.nama_2011 = n;; Assign nilai parameter n ke variabel instance nama_2011.

this.keluhan_2011 = k;; Assign nilai parameter k ke variabel instance keluhan_2011.

this.next_2011 = null;; Penting: Inisialisasi pointer next dengan null, menandakan node baru ini belum terhubung ke node lain (masih berdiri sendiri atau sebagai tail).

5. Metode Setter setN_2011

public void setN_2011(String n): Metode publik untuk mengubah/update nilai nama pasien setelah objek dibuat.

this.nama_2011 = n;; Assign nilai parameter baru ke atribut nama_2011.

Memungkinkan mutasi data nama tanpa perlu membuat objek pasien baru.

6. Metode Setter setK_2011

public void setK_2011(String k): Metode publik untuk mengubah/update nilai keluhan pasien.

this.keluhan_2011 = k;; Assign nilai parameter baru ke atribut keluhan_2011.

Berguna jika ada revisi diagnosa atau penambahan informasi keluhan.

7. Metode Setter setA_2011

public void setA_2011(int a): Metode publik untuk mengubah/update nomor antrian pasien.

this.Antrian_2011 = a;; Assign nilai parameter baru ke atribut Antrian_2011.

Berguna jika terjadi penomoran ulang atau prioritas antrian berubah.

8. Metode Getter getN_2011

public String getN_2011(): Metode publik untuk mengakses nilai nama pasien dari luar kelas.

return nama_2011;; Mengembalikan nilai String dari atribut nama_2011.

Menerapkan prinsip enkapsulasi: data privat diakses melalui metode publik.

9. Metode Getter getK_2011

public String getK_2011(): Metode publik untuk mengakses nilai keluhan pasien.

return keluhan_2011;; Mengembalikan nilai String dari atribut keluhan_2011.

10. Metode Getter getA_2011

public int getA_2011(): Metode publik untuk mengakses nilai nomor antrian pasien.

return Antrian_2011;; Mengembalikan nilai integer dari atribut Antrian_2011.

```

package pekan5_2511532011;

import java.util.Scanner;

public class RumahSakit_2511532011 {
    private static int count_2011 = 0;

    public static Pasien_2511532011 DaftarkanPasien_2011(Pasien_2511532011 item_2011, String n_2011, String
k_2011 ) {
        count_2011++;
        Pasien_2511532011 node_2011 = new Pasien_2511532011(count_2011, n_2011, k_2011 );

        if (item_2011 == null ) {
            return node_2011;
        }

        Pasien_2511532011 l_2011 = item_2011;

        while (l_2011.next_2011 != null) {
            l_2011 = l_2011.next_2011;
        }

        l_2011.next_2011 = node_2011;
        return item_2011;
    }

    public static Pasien_2511532011 PanggilPasien_2011(Pasien_2511532011 item_2011) {
        if (item_2011 == null ) {
            return null;
        }

        System.out.println("Dipanggil Pasien");
        System.out.println("No Antrian: " + item_2011.getA_2011());
        System.out.println("Nama: " + item_2011.getN_2011());
        System.out.println("Keluhan/Penyakit: " + item_2011.getK_2011());
        System.out.println("");

        item_2011 = item_2011.next_2011;
        return item_2011;
    }

    public static void TampilkanAntrian_2011(Pasien_2511532011 item_2011) {
        Pasien_2511532011 c_2011 = item_2011;
        int posisi_2011 = 1;

        while (c_2011 != null) {
            System.out.println("Posisi " + posisi_2011);
            System.out.println("No Antrian: " + c_2011.getA_2011());
            System.out.println("Nama: " + c_2011.getN_2011());
            System.out.println("Keluhan/Penyakit: " + c_2011.getK_2011());
            System.out.println("");
            c_2011 = c_2011.next_2011;
            posisi_2011++;
        }
    }

    public static boolean CariPasien_2011(Pasien_2511532011 item_2011, String n_2011) {
        Pasien_2511532011 c_2011 = item_2011;
        while (c_2011 != null) {
            if (c_2011.nama_2011.equalsIgnoreCase(n_2011)) {
                System.out.println("No Antrian: " + c_2011.getA_2011());
                System.out.println("Nama: " + c_2011.getN_2011());
                System.out.println("Keluhan/Penyakit: " + c_2011.getK_2011());
                System.out.println("");
                return true;
            }
            c_2011 = c_2011.next_2011;
        }
        System.out.println("tidak ada pasien dengan nama yang kamu inputkan");
        return false;
    }

    public static void CekStatusAntrian_2011(Pasien_2511532011 item_2011) {
        if ( item_2011 == null) {
            System.out.println("Antrian Kosong");
            return;
        }

        int ttl_2011 = 0;
        Pasien_2511532011 c_2011 = item_2011;
        while (c_2011 != null) {
            ttl_2011++;
            c_2011 = c_2011.next_2011;
        }

        System.out.println("Jumlah Pasien: " + ttl_2011);
        System.out.println("Pasien Terdepan");
        System.out.println("No Antrian: " + item_2011.getA_2011());
        System.out.println("Nama: " + item_2011.getN_2011());
        System.out.println("Keluhan/Penyakit: " + item_2011.getK_2011());
        System.out.println("");
    }
}

```



```

public static void main(String[] args) {
    Scanner scanner_2011 = new Scanner(System.in);
    Pasien_2511532011 o_2011 = null;

    int pilihan_2011 = 0;

    do {
        System.out.println("\n=== Antrian Rumah Sakit Nim:2511532011 ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        if (scanner_2011.hasNextInt()) {
            pilihan_2011 = scanner_2011.nextInt();
            scanner_2011.nextLine();
        } else {
            System.out.println("Input tidak valid. Masukkan angka.");
            scanner_2011.nextLine();
            continue;
        }

        switch (pilihan_2011) {
            case 1:
                System.out.print("Masukkan nama pasien: ");
                String nama_2011 = scanner_2011.nextLine();
                System.out.print("Masukkan keluhan: ");
                String penyakit_2011 = scanner_2011.nextLine();

                o_2011 = DaftarkanPasien_2011(o_2011, nama_2011, penyakit_2011);
                System.out.print("Pasien berhasil didaftarkan dengan no. antrian: " + count_2011 + "\n");
                break;

            case 2:
                o_2011 = PanggilPasien_2011(o_2011);
                break;

            case 3:
                TampilkanAntrian_2011(o_2011);
                break;

            case 4:
                System.out.print("Masukkan nama pasien yang dicari: ");
                String n_2011 = scanner_2011.nextLine();
                CariPasien_2011(o_2011, n_2011);
                break;

            case 5:
                CekStatusAntrian_2011(o_2011);
                break;

            case 6:
                System.out.println("Program selesai");
                break;

            default:
                System.out.println("Pilihan tidak valid.");
                break;
        }
    } while (pilihan_2011 != 6);

    scanner_2011.close();
}

```

Pembahasan Class Rumah Sakit:

1. Deklarasi Kelas

- public class RumahSakit_2511532011: Kelas publik yang berfungsi sebagai driver utama untuk sistem manajemen antrian pasien berbasis linked list.

2. Variabel Statis Counter

- `private static int count_2011 = 0;`: Variabel statis yang berfungsi sebagai generator nomor antrian otomatis.
- `static`: Nilai inishared di seluruh kelas dan tidak terikat pada instance objek, memastikan nomor antrian tetapurut meskipun metode dipanggil berkali-kali.
- `private`: Akses dibatasi hanya di dalam kelas ini untuk menjaga integritas penomoran.

3. Metode DaftarkanPasien_2011 (Enqueue)

- `public static Pasien_2511532011 DaftarkanPasien_2011(...)`: Metode untuk mendaftarkan pasien baru ke akhir antrian (operasi insert tail).
- `count_2011++`:: Increment counter global untuk menghasilkan nomor antrian unik berikutnya.
- `Pasien_2011 node_2011 = new Pasien_2511532011(count_2011, n_2011, k_2011);`:: Membuat node pasien baru dengan nomor antrian, nama, dan keluhan yang diinput.
- `if (item_2011 == null) { return node_2011; }`: Handle kasus antrian kosong. Node baru langsung menjadi head dan dikembalikan.
- `Pasien_2511532011 L_2011 = item_2011;`:: Inisialisasi pointer traversing L_2011 dimulai dari head.
- `while (L_2011.next_2011 != null) { L_2011 = L_2011.next_2011; }`: Loop untuk menemukan node terakhir (tail) dalam list.
- `L_2011.next_2011 = node_2011;`:: Hubungkan tail saat ini ke node baru, sehingga node baru menjadi tail yang baru.
- `return item_2011;`:: Kembalikan referensi head karena posisi head tidak berubah.
- Kompleksitas: $O(n)$ karena harus traversing seluruh list untuk mencapai tail.

4. Metode PanggilPasien_2011 (Dequeue)

- `public static Pasien_2511532011 PanggilPasien_2011(...)`: Metode untuk melayani pasien terdepan dan menghapusnya dari antrian (operasi delete head).

- `if (item_2011 == null) { return null; }`: Handle kasus antrian kosong, tidak ada yang bisa dipanggil.
- Blok `System.out.println(...)`: Menampilkan informasi lengkap pasien yang dipanggil (nomor antrian, nama, keluhan) menggunakan metode `getter`.
- `item_2011 = item_2011.next_2011;`: Logika Penghapusan. Pindahkan referensi `head` ke node berikutnya. Node lama terisolasi dan akan dihapus oleh garbage collector.
- `return item_2011;`: Kembalikan referensi `head` yang baru (pasien berikutnya dalam antrian).

5. Metode TampilkanAntrian_2011 (Display All)

- `public static void TampilkanAntrian_2011(...)`: Metode untuk mencetak seluruh isi antrian secara berurutan dari depan ke belakang.
- `Pasien_2511532011 c_2011 = item_2011;`: Inisialisasi pointer `c_2011` untuk traversing mulai dari `head`.
- `int posisi_2011 = 1;`: Variabel counter untuk menampilkan nomorurut posisi dalam antrian (1-based).
- `while (c_2011 != null)`: Loop berjalan selama masih ada node yang belum diakses.
- Blok `System.out.println(...)`: Mencetak detail setiap pasien (posisi, nomor antrian, nama, keluhan) dengan format yang rapi.
- `c_2011 = c_2011.next_2011;`: Geser pointer ke node berikutnya.
- `posisi_2011++`: Increment counter posisi untuk pasien berikutnya.

6. Metode CariPasien_2011 (Search by Name)

- `public static boolean CariPasien_2011(...)`: Metode untuk mencari pasien berdasarkan nama dalam antrian.
- `Pasien_2511532011 c_2011 = item_2011;`: Inisialisasi pointer traversing dari `head`.
- `while (c_2011 != null)`: Loop linear search melalui seluruh elemen list.
- `if (c_2011.nama_2011.equalsIgnoreCase(n_2011))`: Logika Pencarian. Membandingkan nama pasien dengan input menggunakan

`equalsIgnoreCase` agar pencarian tidak sensitif terhadap huruf besar/kecil.

- Jika ditemukan: Cetak detail pasien dan return true untuk menghentikan pencarian (early return).
- `c_2011 = c_2011.next_2011;` Lanjut ke node berikutnya jika tidak cocok.
- `System.out.println("tidak ada pasien..."); return false;` Jika loop selesai tanpa menemukan kecocokan, tampilkan pesan tidak ditemukan.

7. Metode `CekStatusAntrian_2011` (Queue Status)

- `public static void CekStatusAntrian_2011(...):` Metode untuk menampilkan ringkasan status antrian saat ini.
- `if (item_2011 == null) { System.out.println("Antrian Kosong"); return; }` Handle kasus antrian kosong.
- `int ttl_2011 = 0;` Inisialisasi counter untuk menghitung total pasien.
- `while (c_2011 != null) { ttl_2011++; c_2011 = c_2011.next_2011; }` Loop traversing untuk menghitung jumlah node dalam list.
- `System.out.println("Jumlah Pasien: " + ttl_2011);` Tampilkan total antrian.
- Blok cetak "Pasien Terdepan": Menampilkan detail pasien yang berada di posisi head (yang akan dipanggil berikutnya).

8. Metode main - Inisialisasi

- `Scanner scanner_2011 = new Scanner(System.in);` Aktifkan scanner untuk input konsol.
- `Pasien_2511532011 o_2011 = null;` Inisialisasi referensi head antrian dengan null, menandakan antrian awal kosong.
- `int pilihan_2011 = 0;` Variabel penampung pilihan menu dari pengguna.

9. Metode main - Loop Menu Interaktif

- `do { ... } while (pilihan_2011 != 6);` Struktur loop yang memastikan menu tampil minimal sekali dan berulang hingga pengguna memilih opsi 6 (Keluar).
- Blok `System.out.println(...):` Mencetak header dan opsi menu (1-6) untuk panduan pengguna.

- Validasi Input Pengguna
- `if (scanner_2011.hasNextInt())`: Validasi tipe input. Mencegah program crash jika pengguna memasukkan huruf/symbol вместо angka.
- `pilihan_2011 = scanner_2011.nextInt(); scanner_2011.nextLine();`: Baca angka pilihan dan konsumsi karakter newline tersisa agar input string berikutnya tidak terlewat.
- Blok `else`: Menangani input tidak valid dengan pesan error dan `continue` untuk mengulang loop.
- Switch Case - Menu 1: Daftarkan Pasien
- `case 1::` Prompt input nama dan keluhan pasien menggunakan `scanner_2011.nextLine()`.
- `o_2011 = DaftarkanPasien_2011(o_2011, nama_2011, penyakit_2011);`: Panggil metode `enqueue`. Penting: Hasil return harus di-assign kembali ke `o_2011` karena head bisa berubah (saat antrian awal kosong).
- Cetak konfirmasi dengan nomor antrian dari variabel `count_2011`.
- Switch Case - Menu 2: Panggil Pasien
- `case 2:: o_2011 = PanggilPasien_2011(o_2011);`: Panggil metode `dequeue`. Head bergeser ke pasien berikutnya, pasien lama terhapus dari antrian.
- Switch Case - Menu 3: Tampilkan Antrian
- `case 3:: TampilkanAntrian_2011(o_2011);`: Panggil metode `display` untuk mencetak seluruh antrian tanpa mengubah struktur data.
- Switch Case - Menu 4: Cari Pasien
- `case 4::` Prompt input nama pencarian, lalu `CariPasien_2011(o_2011, n_2011);`: Jalankan linear search dan tampilkan hasil jika ditemukan.
- Switch Case - Menu 5: Cek Status
- `case 5:: CekStatusAntrian_2011(o_2011);`: Tampilkan ringkasan total pasien dan detail pasien terdepan.
- Switch Case - Menu 6 & Default
- `case 6::` Cetak pesan terminasi program.
- `default::` Tangani angka di luar range 1-6 dengan pesan "Pilihan tidak valid".

10. Terminasi Program

- `scanner_2011.close();`: Menutup resource scanner setelah loop selesai.

Praktik terbaik untuk mencegah resource leak.

Output:

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan nama pasien: udin
Masukkan keluhan: a
Pasien berhasil didaftarkan dengan no. antrian: 1
```

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan nama pasien: surya
Masukkan keluhan: b
Pasien berhasil didaftarkan dengan no. antrian: 2
```

```
<terminated> RumahSakit_2511532011 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (May 7, 2026, 9:19:08PM - 9:20:11PM elapsed: 0:01:03.576) [pid: 21788]
```

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan nama pasien: beni
Masukkan keluhan: c
Pasien berhasil didaftarkan dengan no. antrian: 3
```

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi 1
No Antrian: 1
Nama: udin
Keluhan/Penyakit: a
```

<terminated> RumahSakit_2511532011 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (May 7, 2026, 9:19:08 PM - 9:20:11 PM elapsed: 0:01:03.578) [pid: 21788]

```
Posisi 2
No Antrian: 2
Nama: surya
Keluhan/Penyakit: b

Posisi 3
No Antrian: 3
Nama: beni
Keluhan/Penyakit: c

=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Dipanggil Pasien
No Antrian: 1
Nama: udin
Keluhan/Penyakit: a
```

<terminated> RumahSakit_2511532011 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (May 7, 2026, 9:19:08 PM - 9:20:11 PM elapsed: 0:01:03.578) [pid: 21788]

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien yang dicari: beni
No Antrian: 3
Nama: beni
Keluhan/Penyakit: c

=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah Pasien: 2
Pasien Terdepan
No Antrian: 2
```

Activate Windows

```
=== Antrian Rumah Sakit Nim:2511532011 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai
```


BAB III

KESIMPULAN

3.1 Kesimpulan

ADT LinkedList merupakan struktur data linear berbasis node yang saling terhubung melalui referensi, sehingga memungkinkan pengelolaan data secara dinamis tanpa memerlukan alokasi memori yang bersebelahan. Dalam bahasa pemrograman Java, LinkedList diimplementasikan sebagai doubly-linked list yang menyediakan metode utama seperti add(), remove(), get(), dan set() untuk manipulasi elemen secara terstruktur. Mekanisme kerja LinkedList mengutamakan efisiensi pada operasi penyisipan dan penghapusan karena tidak memerlukan penggeseran elemen, meskipun hal ini mengakibatkan kompleksitas waktu linear saat melakukan akses acak. Karakteristik tersebut menjadikan LinkedList solusi ideal untuk berbagai skenario komputasi yang membutuhkan fleksibilitas data, seperti manajemen riwayat navigasi, representasi graf melalui adjacency list, serta sistem playlist dan cache dinamis. Melalui praktikum ini, pemahaman konseptual mengenai struktur data non-kontigu serta kemampuan teknis dalam memanfaatkan metode LinkedList pada Java telah tercapai secara optimal sebagai fondasi pengembangan aplikasi yang efisien.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:29-april-2026]

